

**AI-POWERED ENTERPRISE MEETING
INTELLIGENCE AND ACTION TRACKING
PLATFORM**

PHASE I REPORT

Submitted by

GIRIJA S P- 230701092

BHUVANESHWARI K- 230701055

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

September 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled "**AI-POWERED ENTERPRISE MEETING INTELLIGENCE AND ACTION TRACKING PLATFORM**" is the bonafide work of **GIRIJA S P (230701092), BHUVANESHWARI K(230701055)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE

Dr. J. Manoranjini M.E., Ph.D.,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai - 602 105.

SIGNATURE

Dr. P. Muneeshwari M.E., Ph.D.,
Associate Professor
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai - 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Organizations conduct a large number of meetings in which decisions, discussion points, assigned responsibilities, and deadlines are communicated through natural spoken interaction. Conventional meeting-minutes preparation requires a participant to listen carefully, take notes, identify important outcomes, and later convert those notes into a structured record. This process can become difficult when employees are travelling, working in noisy environments, or unable to maintain complete notes during the meeting. The proposed AI-Powered Enterprise Meeting Intelligence and Action Tracking Platform addresses this problem by accepting a meeting audio or video recording and applying an AI-based analysis workflow to organize the information contained in the recording. The current implementation is a full-stack web application consisting of a React and Vite frontend, a Node.js and Express.js backend, a REST API layer, Multer-based file handling, and integration with the Google Gemini API through the `@google/genai` package. The frontend runs locally on port 5173 and communicates with the backend running locally on port 5000. A meeting recording is uploaded from the frontend, received by the backend, temporarily stored, uploaded to the configured Gemini service, and analysed using a structured instruction that requests meeting-intelligence fields. The generated response is returned in structured JSON form and stored in a local results file for subsequent retrieval and display.

The implemented output contains a meeting summary, key discussion points, decisions, action items, task assignees, due dates, and deadlines. The architecture therefore converts an unstructured meeting recording into a structured representation that can be displayed through the application interface. The present phase does not claim real-time transcription, online-meeting integration, speaker identification, automatic reminders, multilingual processing, or cloud deployment because these capabilities are not part of the implementation information provided for this report. Similarly, no accuracy percentage, user-study result, or benchmark score is reported without an actual evaluation dataset and measurement. The main contribution of the project is the integration of meeting-recording ingestion, backend API processing, multimodal AI analysis, structured result generation, local result storage, and a web-based presentation layer into a single enterprise-oriented workflow. The system provides a practical foundation for future extensions such as real-time transcription, calendar and email integration, task reminders, enterprise authentication, intelligent meeting search, multilingual processing, speaker identification, and large-scale evaluation.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N.S MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. Manoranjini, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, Dr. P. Muneeshwari, Associate Professor, Department of Computer Science and Engineering for her valuable guidance throughout the course of the project. We thank our Project Coordinator, Mr. Madhusudhan, B.E., M.E., Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

GIRIJA S P 230701092

BHUVANESHWARI K 230701055

TABLE OF CONTENTS

ABSTRACT	iii
LIST OF TABLES	ix
LIST OF FIGURES	x
LIST OF ABBREVIATIONS	xii
1. INTRODUCTION	1
1.1 GENERAL	1
1.2 OBJECTIVE	2
1.3 EXISTING SYSTEM	3
1.4 PROPOSED SYSTEM	3
2. LITERATURE SURVEY	5
2.1 LITERATURE REVIEW	5
2.2 COMPARISON AND DISCUSSION	8
2.2.1 RESEARCH GAP	9
2.2.2 PROPOSED CONTRIBUTION/NOVELTY	9
2.3 RESEARCH QUESTIONS	10
2.4 CONCLUSION	10
3. SYSTEM DESIGN	11
3.1 SYSTEM ARCHITECTURE	11
3.2 SYSTEM REQUIREMENTS	12
3.2.1 SOFTWARE REQUIREMENTS	12
3.2.2 HARDWARE REQUIREMENTS	13
3.3 API DESIGN	13

3.4 DEPLOYMENT ARCHITECTURE	14
4. PROJECT DESCRIPTION	15
4.1 METHODOLOGY	15
4.2 MODULES	15
4.2.1 USER / MEETING MANAGEMENT MODULE	15
4.2.2 MEETING RECORDING UPLOAD MODULE	16
4.2.3 AUDIO / VIDEO PROCESSING MODULE	18
4.2.4 AI MEETING ANALYSIS MODULE	19
4.2.5 MEETING SUMMARY MODULE	19
4.2.6 DECISION EXTRACTION MODULE	20
4.2.7 ACTION ITEM EXTRACTION MODULE	20
4.2.8 DEADLINE AND DUE-DATE EXTRACTION MODULE	21
4.2.9 MEETING RESULTS MODULE	21
4.2.10 MEETING HISTORY AND TRACKING MODULE	21
5. IMPLEMENTATION AND RESULT DISCUSSION	24
5.1 FRONTEND IMPLEMENTATION	24
5.2 BACKEND IMPLEMENTATION	24
5.3 API ENDPOINTS	23
5.4 GEMINI AI INTEGRATION	26
5.5 MEETING RECORDING UPLOAD	26
5.6 AI PROCESSING WORKFLOW	27
5.7 END-TO-END SYSTEM DEMONSTRATION	27
5.8 FUNCTIONAL TESTING	29

5.9 SYSTEM OUTPUTS	30
5.10 OBSERVED ISSUES	32
5.11 LIMITATIONS	33
5.12 SUMMARY OF CONTRIBUTIONS	33
6. CONCLUSION AND FUTURE WORK	35
6.1 CONCLUSION	35
6.2 FUTURE WORK	35
REFERENCES	36

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
3.1	Software requirements	13
3.2	Hardware requirements	13
3.3	Input and output description	14
4.1	Module implementation	23
5.1	API endpoint summary	26
5.2	Functional test cases	31
5.3	Observed issues and corrective considerations	32

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	Overall System Architecture	11
5.1	Meeting Upload Screen	28
5.2	AI Processing Screen	28
5.3	Meeting Summary	29
5.4	Action Items	29

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
CSS	Cascading Style Sheets
FF	Frontend
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
REST	Representational State Transfer
SPA	Single Page Application
STT	Speech-to-Text
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

CHAPTER 1 INTRODUCTION

1.1 GENERAL

Meetings are a fundamental mechanism through which organizations coordinate projects, distribute responsibilities, review progress, resolve issues, and make decisions. A single meeting may contain several types of information at the same time. Participants can discuss a problem, agree on a solution, assign a task to an employee, specify a target date, and introduce follow-up work within a short sequence of conversation. When this information remains only in the original recording or in incomplete personal notes, it can become difficult to retrieve and act upon later.

The difficulty is increased when employees are not in an ideal note-taking environment. An employee may participate while travelling, work in a noisy location, or need to concentrate on the discussion instead of continuously writing minutes. Network instability can also affect online meeting participation. These conditions create a practical need for preserving the meeting content and transforming it into an organized representation that can be reviewed after the meeting.

The AI-Powered Enterprise Meeting Intelligence and Action Tracking Platform is designed around this requirement. The system accepts a meeting audio or video recording and passes the recording through a backend processing pipeline. The backend temporarily stores the uploaded media, transfers it to the configured Gemini service, and requests structured meeting intelligence. The returned information is organized into summary, key discussion points, decisions, action items, assignees, due dates, and deadlines. The frontend then presents the generated result to the user.

The current phase should be understood as a recording-based meeting analysis system rather than a real-time meeting assistant. The implementation supplied for this report does not include live transcription, direct integration with online meeting platforms, speaker identification, automated reminders, or calendar and email synchronization. Those capabilities are considered future extensions rather than current functionality.

1.2 OBJECTIVE

The objective of the project is to develop an enterprise-oriented web platform that can transform a recorded meeting into a structured set of information that is easier to review and follow up.

1. Provide a web interface through which a user can submit a meeting audio or video recording.
2. Transfer the uploaded recording from the React frontend to the Node.js and Express.js backend through a REST-based communication layer.
3. Process the recording using the configured Google Gemini API through the `@google/genai` package.
4. Generate a structured meeting result containing a summary, key discussion points, decisions, action items, assignees, due dates, and deadlines.
5. Store the generated meeting result in the current local result-storage mechanism so that the frontend can retrieve and display it.
6. Provide a modular architecture that can be extended later with real-time transcription, enterprise integrations, task reminders, authentication, search, and cloud deployment.

1.3 EXISTING SYSTEM

The existing approach in many meeting environments is based on manual note taking or basic recording storage. In a manual approach, one or more participants record important points while the meeting is taking place. After the meeting, the notes may be edited into minutes and distributed to participants. The quality of the final record depends on the attention, experience, and availability of the person taking notes.

Another approach is to store the complete audio or video recording. Recording preserves the source information but does not by itself organize the content into decisions or tasks. A participant who needs to recover one action item must search through the recording manually. A third approach is to use separate transcription or summarization tools. Such tools can reduce the effort of producing text, but the complete workflow still requires identifying which parts correspond to decisions, actions, assignees, and deadlines.

1.4 PROPOSED SYSTEM

The proposed system introduces an AI-based processing pipeline between the meeting recording and the final meeting record. The user first uploads a meeting audio or video recording through the React frontend. The frontend sends the media to the Express backend. Multer handles the multipart upload and the backend temporarily stores the recording. The backend then uploads the recording to Gemini using the `@google/genai` package and sends an analysis instruction describing the required output structure.

Gemini processes the supplied media and returns the meeting analysis. The backend expects the result to contain the defined fields: summary, keyPoints, decisions, actionItems, and deadlines. Each action item contains task, assignee, and dueDate fields. The backend stores the generated result in the current data-storage

mechanism, and the frontend retrieves and renders the structured information for the user.

The architecture is intentionally divided into presentation, application, AI-service, and storage responsibilities. This separation makes the system easier to modify. For example, a future version could replace the current local storage with a database without changing the overall frontend-to-AI workflow. Similarly, future speech-specific preprocessing could be introduced before the Gemini analysis stage.

CHAPTER 2 LITERATURE SURVEY

This chapter reviews research and technical literature relevant to the proposed meeting-intelligence platform. The review covers meeting corpora and meeting understanding, automatic speech recognition, Transformer-based language processing, meeting summarization, actionable-item extraction, and the use of large language models for real-world meeting summarization. The purpose is not to claim that the project reproduces any of the reviewed research, but to establish the technical context in which the current system is positioned.

2.1 LITERATURE REVIEW

Existing research has demonstrated the growing use of LLM-based agents for complex task automation. Zhu et al. surveyed the LLM-based multi-agent orchestration landscape, categorizing approaches into centralized, decentralized, and hierarchical methods to coordinate LLM agents [1]. LangGraph enables a graph-based approach to control stateful and controlled agent workflows [2]. Duan and Wang explored combining LangGraph with CrewAI for multi-agent task orchestration [3].

The development of intelligent document processing has come a long way too. Gómez and Sánchez studied LLM-based extraction of structured data from invoices and emphasized the role of prompting and proper structure of the documents [4]. Yashwant et al. suggested evaluation measures for invoice information extraction include field accuracy and exact match [5]. Berghaus et al. conducted research comparing multimodal vision-based and purely textual invoice processing approaches and stressing the importance of choosing appropriate document-processing approaches [6]. PaddleOCR 3.0 expands OCR capabilities toward document parsing and key information extraction [12], and Nagayi et al. show that

OCR output on imperfectly captured text must be evaluated rather than assumed to be reliable [13].

Human intervention is also a critical aspect in the context of AI-driven decision-making systems. Adedokun et al. reviewed human-in-the-loop mechanisms for agentic AI [10], while Cheng and Cheng proposed a decoupled architecture for controlled human intervention in agentic workflows [11]. Research in healthcare and insurance claims has additionally demonstrated the application of machine learning to automated claim analysis and fraud detection [7], [8], [9].

Srinivasan et al. proposed an optimized warranty analysis methodology for the fleet industry using data mining, clustering, and pattern-recognition techniques to identify anomalous and potentially fraudulent warranty claims [14]. Their approach analyzes large volumes of historical warranty and component-failure data and considers factors such as failure date, claim date, and failure mode to identify suspicious claims for further review. The study also integrates warranty claims data with business rules, reports, and claim-processing information, demonstrating the value of data-driven analytics in improving warranty claim analysis.

However, the reviewed studies largely address these areas independently. There is limited integration of document extraction, product verification, warranty-policy checking, agent-based reasoning, and human review into a unified warranty claim validation workflow. Therefore, the proposed system focuses on integrating these capabilities into an agentic AI-driven warranty claim validation system.

2.2 COMPARISON AND DISCUSSION

Read together, the reviewed literature converges on three major aspects that shaped the design of the proposed AI-powered enterprise meeting intelligence and action

tracking platform. First, meeting-understanding research demonstrates that recorded meetings contain multiple types of information, including spoken discussions, decisions, and task-related information. The AMI Meeting Corpus established an important research context for computational meeting understanding using multimodal meeting recordings and annotations, while speech-recognition research such as Whisper and wav2vec 2.0 demonstrates the importance of reliable processing of spoken content. These studies provide the foundation for processing recorded meeting audio and video in the proposed system. Second, research on Transformer-based language models, BERT, and meeting summarization demonstrates the effectiveness of modern language-processing approaches for extracting meaningful information from lengthy conversational content. QMSum specifically highlights the difficulty of summarizing long and information-rich meetings, while more recent LLM-based meeting summarization research demonstrates the potential of large language models for practical meeting analysis. These findings directly support the use of a multimodal generative AI model in the proposed system to generate meeting summaries and key discussion points. Third, research on actionable meeting information shows that identifying tasks from conversations is an important part of meeting understanding. The AIMU work specifically addresses actionable-item detection, which is closely related to the proposed system's extraction of action items, responsible persons, and due dates. The Gemini technical literature and API documentation further support the use of multimodal AI for analysing recorded audio and video content.

Where the reviewed works differ most from the proposed system is in their scope of integration and application workflow. The AMI Meeting Corpus is primarily a research dataset and evaluation resource rather than an enterprise meeting-management application. Whisper and wav2vec 2.0 primarily address speech

recognition and speech representation learning, rather than providing a complete workflow for meeting analysis and action tracking. Transformer and BERT research provides the underlying foundations for language processing but does not itself implement an enterprise meeting-management platform. QMSum focuses on meeting summarization as a benchmark and research task, while AIMU concentrates on identifying actionable items from meetings. Recent LLM-based meeting summarization studies investigate the practical use of large language models for meeting understanding, but they do not represent the exact frontend-to-backend workflow implemented in this project. In contrast, the proposed platform integrates meeting recording upload, backend processing, multimodal AI analysis, structured result generation, and presentation of summaries, key points, decisions, action items, assignees, and deadlines within a single enterprise-oriented workflow. Therefore, the contribution of the current project is not the development of a new speech-recognition or language model, but the integration of existing AI capabilities into an application that converts recorded meeting content into structured and actionable meeting information.

This comparison establishes the research gap addressed by the proposed system: existing literature provides strong foundations for speech recognition, meeting summarization, actionable-item extraction, and multimodal language understanding, but these capabilities are commonly studied as individual research problems or evaluation tasks. The proposed project brings these capabilities together into an integrated meeting-intelligence workflow intended to support practical enterprise meeting management and reduce the manual effort required to identify important discussions, decisions, assigned tasks, and deadlines.

2.2.1 RESEARCH GAP

The reviewed literature indicates strong progress in individual components of meeting understanding, including speech recognition, meeting summarization, and actionable-item detection. However, the cited research is generally centered on a particular model, dataset, benchmark, or task. The current project addresses an application-level integration gap by combining recording upload, backend processing, multimodal AI analysis, structured JSON output, local result storage, and a web presentation layer in one workflow.

The identified gap is therefore not the absence of meeting summarization research. Instead, it is the practical integration of several capabilities into a small enterprise-oriented application that produces explicit fields for summary, key points, decisions, actions, assignees, due dates, and deadlines. The current implementation provides this integration without claiming a new speech-recognition or language-model algorithm.

2.2.2 PROPOSED CONTRIBUTION/NOVELTY

The contribution of the project is primarily system-level. First, it provides an end-to-end recording-to-meeting-intelligence workflow. Second, it requests a structured output rather than an unrestricted paragraph so that downstream application components can display each information category independently. Third, it combines a React frontend, Express backend, Multer-based file processing, Gemini multimodal analysis, and local result storage into a single meeting-management application. The novelty should therefore be described as an integration and application contribution rather than as a new machine-learning algorithm.

2.3 RESEARCH QUESTIONS

1. Can a web-based recording ingestion pipeline transfer meeting audio/video to an AI service and return a structured meeting-intelligence result?
2. Can a multimodal large language model be used to organize meeting content into summaries, key points, decisions, action items, assignees, due dates, and deadlines?
3. Can a modular React–Express architecture provide a practical interface for retrieving and presenting structured meeting results?
4. What implementation limitations and extension requirements become visible when a recording-based meeting-intelligence workflow is used as the foundation for enterprise meeting management?

2.4 CONCLUSION

The literature establishes the technical foundations for the proposed system. Meeting corpora demonstrate the complexity of multiparty meeting data; speech-recognition research provides methods for converting audio into usable information; Transformer and language-model research provides the foundation for contextual language understanding; and meeting-specific research demonstrates the importance of summaries and actionable items. The current project builds on these established areas by integrating multimodal AI analysis into a web-based enterprise meeting workflow. Its present contribution is practical and system-oriented, while more advanced research questions such as multilingual evaluation, speaker attribution, benchmark-based accuracy, and large-scale enterprise validation remain future work.

CHAPTER 3 SYSTEM DESIGN

This chapter describes the architecture and engineering requirements of the AI-Powered Enterprise Meeting Intelligence and Action Tracking Platform. The design follows a client-server approach in which the React frontend provides the user interface, the Node.js and Express.js backend exposes the REST API and manages file processing, the Gemini service performs AI analysis, and the current local JSON-based storage mechanism retains generated meeting results.

3.1 SYSTEM ARCHITECTURE

The system follows a layered architecture. At the presentation layer, a React application built with Vite provides the meeting-management interface and runs on localhost:5173. At the application layer, a Node.js runtime with Express.js provides the REST API and runs on localhost:5000. The backend uses Multer to receive the uploaded meeting recording and temporarily stores the file. The AI-service layer uses the Google Gemini API through the `@google/genai` package. The storage layer currently uses a `data/results.json` file for generated meeting results.

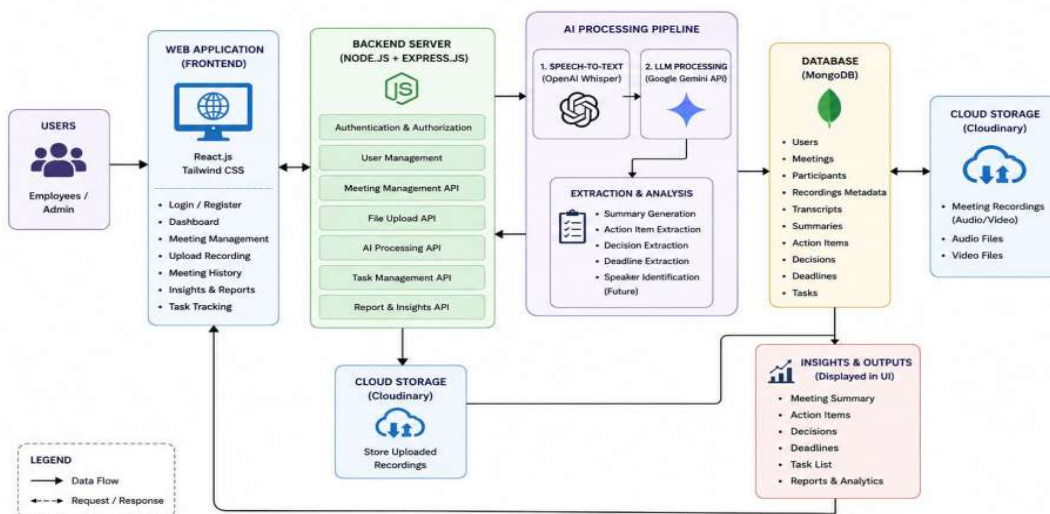


Figure 3.1. System architecture diagram

Figure 3.1 summarises this architecture: the three role-scoped portals on the left; the FastAPI application server in the centre, with its REST routers (authentication, claims, investigator and admin) and its agentic reasoning layer comprising the Intake graph and the Supervisor graph; the external services that layer calls (the configured LLM provider, and PaddleOCR as a local fallback OCR engine); and the MySQL database on the right, grouped into supply-chain and warranty records, claim and validation records, and identity and audit records. The claim-submission endpoint hands each claim to the agent pipeline as an asynchronous background task, which is what allows the customer-facing request to return immediately while validation continues in the background.

3.2 SYSTEM REQUIREMENTS

3.2.1 SOFTWARE REQUIREMENTS

Component	Role
Operating system	Windows development environment can be used for the current local implementation.
Frontend runtime	Node.js environment with React and Vite.
Frontend technologies	React, Vite, JavaScript, HTML, CSS.
Backend runtime	Node.js with Express.js.
API style	REST API for frontend-backend communication.
File upload	Multer for multipart meeting audio/video uploads.
AI service	Google Gemini API
AI SDK	@google/genai package.

Storage	Local JSON result file: data/results.json.
Development tools	A code editor/IDE and terminal/command prompt.
Network	Internet connectivity is required for communication with the Gemini API.

Table 3.1. Software requirements

3.2.2 HARDWARE REQUIREMENTS

Component	Recommendation
Processor	Modern dual-core or better processor
RAM	8 GB or more
Storage	At least 5 GB free for project files, dependencies and uploaded test media
Display	Standard desktop/laptop display suitable for web development
Network	Stable internet connection for Gemini API requests
GPU	Not required by the current application architecture

Table 3.2. Hardware requirements

3.3 API DESIGN

The backend exposes a REST API between the React frontend and the Node.js application. The supplied implementation description confirms that the API accepts a meeting recording, processes it, stores the generated result, and provides the result to the frontend. Exact route names were not supplied in the project information, so the report does not invent endpoint paths. The final version of the report should

replace the placeholders in Chapter 5 with the exact routes from the implemented meetingRoutes.js file.

Operation	Purpose
Meeting upload	Submit an audio/video recording for analysis
Meeting result retrieval	Retrieve an analysed meeting result
Meeting history retrieval	Retrieve previously stored results if supported by current frontend/backend flow
Health/status request	Backend availability or processing status if present

Table 3.3. Input and output description

3.4 DEPLOYMENT ARCHITECTURE

The current implementation is designed for local development and demonstration. The React frontend runs on localhost:5173 and the Node.js/Express backend runs on localhost:5000. The frontend communicates with the backend over HTTP, while the backend communicates with the external Gemini API. Uploaded media is temporarily stored by the backend, and generated results are stored in the local data/results.json file.

The current deployment architecture should not be described as a production cloud deployment. Enterprise deployment, containerization, centralized database storage, authentication, monitoring, secure file storage, and horizontal scaling are appropriate future improvements. The present architecture is sufficient to demonstrate the core recording-to-analysis workflow while keeping the implementation manageable for the current project phase.

CHAPTER 4 PROJECT DESCRIPTION

4.1 METHODOLOGY

The project was developed as a sequence of connected modules around the life cycle of a recorded meeting. The frontend was established first to provide a user-facing meeting interface. The backend was then implemented to receive uploaded media and expose the REST API. File processing was added using Multer, after which the Gemini service integration was connected through `@google/genai`. The AI response was structured into a predictable JSON representation and connected to local result storage. Finally, the frontend was connected to the backend result flow so that the generated meeting information could be displayed.

This methodology emphasizes separation of responsibilities. The frontend is responsible for interaction and presentation; the backend is responsible for request handling, temporary file management, AI-service communication, and result storage; Gemini is responsible for the AI analysis; and the result file provides the current local persistence mechanism. Such separation reduces coupling and provides clear locations for future extensions.

4.2 MODULES

4.2.1 USER / MEETING MANAGEMENT MODULE

The intake module (`app/agents/intake/`) is a three-node LangGraph graph: `extract_pdf_text`, `parse_invoice_text`, and `detect_conflicts`. The first node delegates to `InvoiceExtractionService` (`app/services/invoice_service.py`), which first attempts PyMuPDF text-layer extraction across every page of the uploaded PDF; if the combined extracted text is under 100 characters (the heuristic used to detect a scanned, image-only PDF), it automatically falls back to `PaddleOCRService`, which rasterises each page and runs PaddleOCR's angle-classification and recognition pipeline over it. The second node passes the combined page text to whichever LLM

provider is configured (Google Gemini, model `gemini-2.5-flash`, for every run reported in Chapter 5; `get_llm()` in `app/llm/base.py` checks for an OpenAI, Gemini, then Anthropic API key, in that order, and returns `None` if no key is set, in which case a fixed mock extraction result is used so the rest of the pipeline remains testable without any API key) bound to a structured-output schema, `InvoiceExtractionResult`, so the model can only return the seven typed fields the schema defines and cannot invent extra ones. The third node, `detect_conflicts`, reconciles each of `serial_number`, `customer_name`, `customer_phone` and `invoice_number` between what the customer typed and what the LLM extracted: identical values are accepted as-is, a genuine mismatch is recorded in a conflicts dictionary and forces the claim's `next_step` to `MANUAL_REVIEW`, and a missing required field routes the claim to `MORE_INFORMATION_REQUIRED` before the Supervisor graph is invoked, so a claim never silently proceeds on incomplete or contradictory data.

4.2.2 MEETING RECORDING UPLOAD MODULE

Once the Intake Agent has produced a confirmed serial number, the Supervisor graph (`app/agents/supervisor.py`) runs eight agents in a fixed sequence, each one a thin, single-purpose class that queries the database directly:

1. Customer Verification Agent: confirms a customer row exists whose `serial_number`, `invoice_number` and `customer_phone` all match the claim. If the unit's warranty has since been transferred, it follows the `ownership_transfer` chain for that serial number and accepts the claim only when the claimant matches the latest owner in that chain (the `to_customer_name` and `to_phone` of the most recent transfer), so neither the original buyer after a resale nor an intermediate owner can claim on the warranty.

2. Product Validation Agent: looks up the serial number in the manufacturer table and flags a MISMATCH if the claimed product name or model number differs from the manufacturer's own record, or INVALID if the serial number does not exist at all.
3. Manufacturer → Seller Agent: confirms a manufacturer_to_seller transfer record exists for the serial number, i.e. that the unit legitimately entered the retail channel.
4. Seller → Customer Agent: confirms a seller row exists whose serial_number and invoice_number match.
5. Warranty Validation Agent: looks for an explicit warranty row first; if none exists, it derives one from the seller's sale date plus the manufacturer's warranty_period_months (using dateutil.relativedelta) and compares the claim date against that computed window to return ACTIVE or INACTIVE.
6. Policy Validation Agent: only runs once the warranty is ACTIVE; it loads the applicable Policy row's covered_conditions and excluded_conditions and asks the configured LLM to classify the claim description as COVERED, EXCLUDED, or UNCERTAIN, parsing the model's JSON response defensively and falling back to UNCERTAIN if parsing fails.
7. Claim History Agent: queries a warranty_claim_summary database view for the number of prior claims against the same warranty_id and labels the history risk HIGH once that count exceeds two.
8. Fraud Risk Agent: described in 4.2.3 below.

Each node in the Supervisor's StateGraph reads only the specific prior results it needs from the shared OrchestratorState TypedDict and writes its own result back under its own key, so a later agent (for example, Policy Validation) can short-circuit

cleanly, returning a SKIPPED status with a reason, when an earlier agent (Warranty Validation) has already made its check moot.

4.2.3 AUDIO / VIDEO PROCESSING MODULE

The Fraud Risk Agent (`app/agents/fraud_risk/agent.py`) computes an additive, fully transparent score rather than calling a trained classifier: +40 if customer ownership could not be verified, +50 if the serial number was not found at all, +30 for a product/model mismatch, +20 for a missing manufacturer-to-seller record, +40 for a missing seller-to-customer record, +40 if the warranty is not active, and +20 if the customer has two or more prior approved claims. The resulting score is bucketed into LOW (0-29), MEDIUM (30-59) or HIGH (60+), and every contributing signal is kept as a human-readable string (for example "Serial number not found in manufacturer database (+50)") so an investigator can see exactly why a score was assigned, not just the number itself. The score, risk level and signal list are persisted to the database during background processing, so the investigator claims queue and the operations dashboard read the stored score rather than recomputing it.

The Decision Agent (`app/agents/decision/agent.py`) then applies a short, ordered rule list over every upstream result: an invalid serial number or an inactive warranty is an automatic REJECTED; a claim the Policy Agent classified as EXCLUDED is also REJECTED, quoting the policy agent's own reason; unverifiable customer ownership, an invalid seller relationship, an UNCERTAIN policy classification (including the case where the policy evaluation itself fails to complete), or a HIGH fraud-risk level all route the claim to INVESTIGATION_REQUIRED; and any claim that clears every check is APPROVED. Because eligibility (warranty/policy) and risk (fraud score) are checked as genuinely separate conditions rather than folded into one number, a claim can be fully eligible under warranty and policy terms and

still be sent for investigation purely because its fraud-risk signals were high, which keeps the two concerns auditable independently.

4.2.4 AI MEETING ANALYSIS MODULE

Authentication is stateless JWT-based: `app/core/security.py` issues a 24-hour access token (`create_access_token`) embedding the user's id and role, hashes stored passwords with Argon2 via `passlib`, and exposes three FastAPI dependencies (`get_current_user`, `get_current_investigator`, and `get_current_admin`) that decode the bearer token, load the corresponding user, and raise a 401 or 403 the moment a role check fails. Every investigator and admin route in the codebase depends on one of these guards, so role enforcement happens at the framework level rather than being re-implemented per endpoint. The frontend mirrors this with a lightweight `ProtectedRoute` wrapper that redirects to `/login` whenever no token is present in `localStorage`, and an axios request interceptor (`src/api/client.ts`) that automatically attaches the bearer token to every outgoing API call.

4.2.5 MEETING SUMMARY MODULE

Every administrative create or edit action in the Admin portal (creating a manufacturer, seller, customer, warranty, transfer, or policy, or editing an existing policy) calls `log_audit()` (`app/core/audit.py`), which writes an `AuditLog` row recording the acting user's id and role, the action name, the entity type and id affected, and JSON-serialised snapshots of both the old and new data where applicable. The Admin portal's Audit Logs page surfaces the most recent 100 entries, giving a chronological, tamper-evident record of who changed what across the manufacturer, seller, policy, warranty and transfer registers. Investigator decisions on individual claims are written to the same log (`CLAIM_APPROVED`,

CLAIM_REJECTED), as are administrator actions such as creating investigator accounts.

4.2.6 DECISION EXTRACTION MODULE

Purpose: To identify decisions or agreements contained in the meeting discussion.

Input: Meeting recording processed by Gemini.

Processing: The AI analysis instruction requests decisions as a separate structured field. The returned decisions are stored as part of the meeting result.

Output: A list of identified decisions.

Technologies used: Gemini API, structured JSON, Express.js, React.

Contribution: Separating decisions from the general summary makes important outcomes easier to review.

4.2.7 ACTION ITEM EXTRACTION MODULE

Purpose: To identify tasks that participants need to perform after the meeting.

Input: Meeting recording and AI analysis request.

Processing: Gemini identifies action-oriented statements and returns action items in structured objects. Each object contains task, assignee, and dueDate fields.

Output: A structured action-item list.

Technologies used: Gemini API, JSON, Node.js, React.

Contribution: The module transforms conversational commitments into a form that can support later task-management features.

4.2.8 DEADLINE AND DUE-DATE EXTRACTION MODULE

Purpose: To preserve explicit time-related commitments from the meeting.

Input: Meeting recording and identified action items.

Processing: The AI output includes dueDate values within action items and a separate deadlines field for deadline-related information.

Output: Due dates attached to tasks and meeting-level deadline information.

Technologies used: Gemini API, JSON, Node.js, React.

Contribution: Explicit time information makes action items more useful for follow-up and provides a foundation for future reminder functionality.

4.2.9 MEETING RESULTS MODULE

Purpose: To store and present the complete structured meeting analysis.

Input: JSON returned by Gemini.

Processing: The backend stores the result in data/results.json and the frontend retrieves and displays the individual fields.

Output: A meeting result view containing summary, key points, decisions, action items, assignees, due dates, and deadlines.

Technologies used: Node.js, Express.js, JSON storage, React.

Contribution: This module converts the AI response into a persistent and user-readable application result.

4.2.10 MEETING HISTORY AND TRACKING MODULE

Purpose: To support access to stored meeting results and provide a basis for reviewing analysed meetings.

Input: Stored meeting-result records.

Processing: The backend and frontend use the current result-storage mechanism to retrieve generated information as supported by the implemented application.

Output: Previously generated meeting information where the current implementation exposes it.

Technologies used: Node.js, Express.js, JSON storage, React.

Contribution: The module supports continuity between the initial AI analysis and later review. Advanced task progress tracking and reminders are not claimed as implemented.

Module	Primary input	Primary output	Main technologies
User / Meeting Management	User interaction	Meeting-analysis request/result view	React, Vite
Recording Upload	Audio/video file	Backend-accessible file	React, Express, Multer
Audio / Video Processing	Uploaded media	Gemini input	Node.js, @google/genai
AI Meeting Analysis	Meeting media	Structured JSON	Gemini API
Meeting Summary	AI result	Summary text	Gemini, React
Decision Extraction	AI result	Decision list	Gemini, JSON
Action Item Extraction	AI result	Task/assignee/ dueDate objects	Gemini, JSON
Deadline / Due Date	AI result	Due dates and deadlines	Gemini, JSON

Meeting Results	Stored JSON	Structured meeting display	Express, React
History / Tracking	Stored results	Previous meeting information	JSON, Express, React

Table 4.1. Module implementation summary

CHAPTER 5 IMPLEMENTATION AND RESULT DISCUSSION

This chapter documents the implementation of the current prototype and discusses the results that can be established from the supplied implementation details. Because no measured benchmark dataset, accuracy study, user study, or latency experiment has been supplied, the discussion focuses on implemented functionality, functional test design, observable system outputs, and limitations. Quantitative performance fields are marked [TO BE MEASURED] where actual measurements would be required.

5.1 FRONTEND IMPLEMENTATION

The frontend is implemented as a React application using Vite, JavaScript, HTML, and CSS. It runs on localhost:5173. The frontend provides the user-facing meeting-management workflow and communicates with the backend through the REST API. The interface is responsible for accepting the meeting recording and presenting the generated result in a structured form.

The frontend should be described according to the actual screens available in the current project. In particular, the final report should capture the real upload screen, processing state if one is visible, summary output, action-item output, and decision/deadline output. Screenshots must be taken from the running application and should not be recreated with fabricated sample data.

5.2 BACKEND IMPLEMENTATION

The backend is implemented using Node.js and Express.js and runs on localhost:5000. The project follows a modular structure containing server.js,

controllers, routes, services, uploads, and data directories. The `aiController.js` controller is responsible for the AI-related request handling, `meetingRoutes.js` defines meeting-related API routes, and `geminiService.js` provides the service-layer integration with Gemini. Uploaded files are temporarily handled through the uploads directory, while generated results are stored under `data/results.json`.

This separation between controller, route, service, upload storage, and result storage provides a clear backend flow. A route receives the request, the controller coordinates the operation, the service communicates with Gemini, and the generated result is written to the current storage layer. The exact function names and route paths should be copied from the final source code when the report is finalized.

5.3 API ENDPOINT SUMMARY

The project information confirms the presence of a REST API and a meeting route module but does not provide the exact route strings, HTTP methods, request field names, or response schemas. To avoid inventing implementation details, Table 5.1 intentionally leaves those values as placeholders. Replace each placeholder with the exact value from `meetingRoutes.js` and the corresponding controller before final submission.

Method & Path	Description
<code>http://localhost:5173/upload-recording/2</code>	Upload meeting audio/video
<code>http://localhost:5173/ai-results/5</code>	Retrieve generated meeting result
<code>http://localhost:5173/history</code>	Retrieve previous meeting results

http://localhost:5173/Profile	Show profile details
---	----------------------

Table 5.1. API endpoint summary

5.4 GEMINI AI INTEGRATION

The AI layer uses the Google Gemini API through the `@google/genai` package. The backend uploads the meeting recording to Gemini and supplies an instruction requesting a structured meeting analysis. The design relies on Gemini's multimodal understanding of audio/video rather than implementing a separate local speech-recognition model in the current phase. Google's current documentation describes Gemini audio understanding for tasks including transcription, summarization, and question answering, and video understanding for extracting information from audiovisual content [11], [12].

The report records the configured model exactly as supplied for this project: Gemini 3.6 Flash. Model availability and API behavior can change over time, so the final report should also record the actual model identifier used in the project's environment at the time of submission. No claim is made here about accuracy because no project-specific benchmark has been provided.

5.5 MEETING RECORDING UPLOAD

The recording-upload process is implemented through the React frontend, Express backend, and Multer file-handling layer. The user selects a supported audio or video recording. The frontend sends the recording to the backend. Multer receives the multipart request and stores the file temporarily. The current configured upload limit is 25 MB. The backend then uses the stored file as the input for the Gemini upload step.

The 25 MB limit is an implementation configuration and should be verified directly in the final source code before submission. The report should not describe larger-file support unless the configuration is changed and tested.

5.6 AI PROCESSING WORKFLOW

1. The user uploads the meeting recording from the frontend.
2. The backend receives the multipart request and temporarily stores the file.
3. The backend invokes the Gemini service through `@google/genai`.
4. The meeting recording is supplied to Gemini for analysis.
5. The analysis instruction requests the defined meeting-intelligence fields.
6. Gemini returns structured JSON.
7. The backend stores the result in `data/results.json`.
8. The frontend retrieves and displays the generated meeting information.

5.7 END-TO-END SYSTEM DEMONSTRATION

An end-to-end demonstration should begin with a real meeting recording selected from the upload interface. The demonstration should show the upload action, backend processing, successful AI response, storage of the result, and presentation of the structured meeting information. Each screenshot should be captured from the actual running system and should preserve the application's real interface.

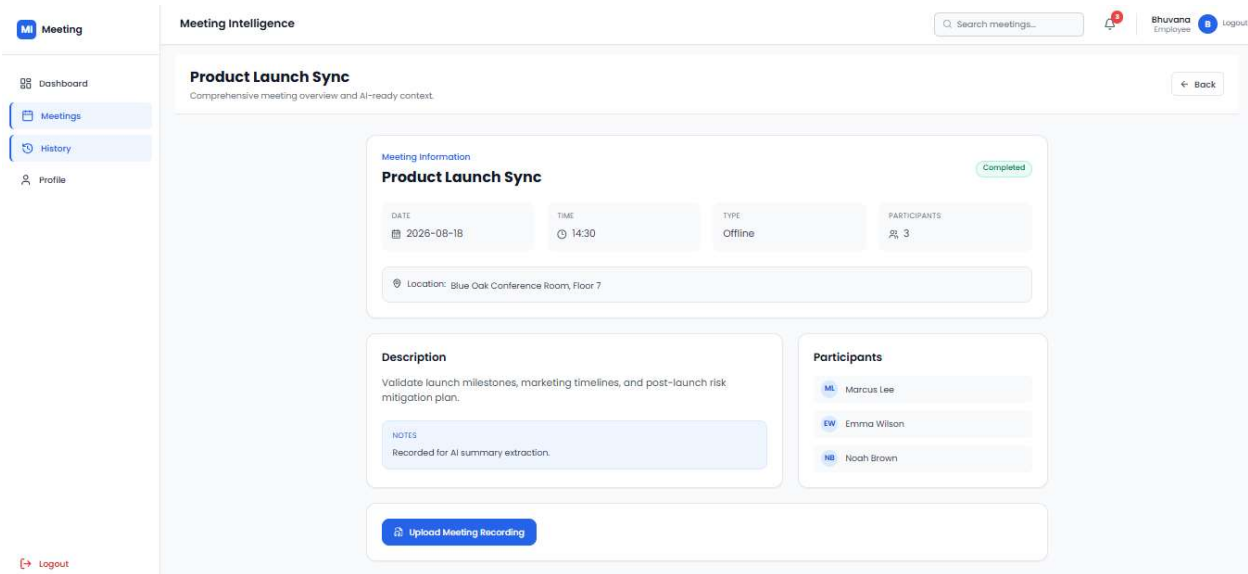


Figure 5.1. Meeting Upload Screen

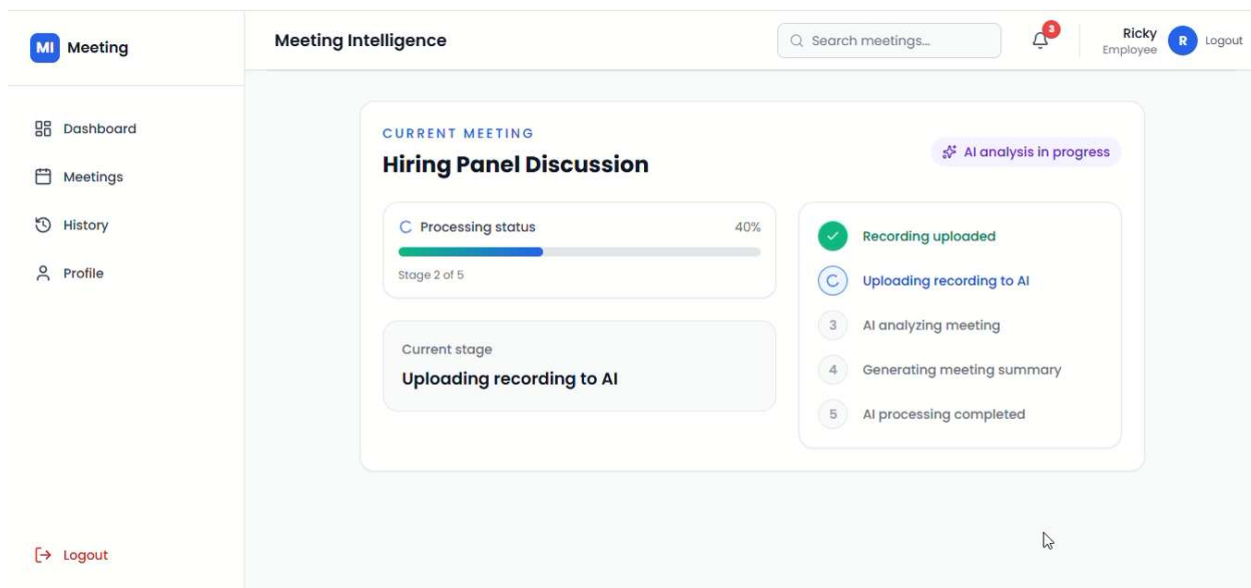


Figure 5.2. AI Processing Screen

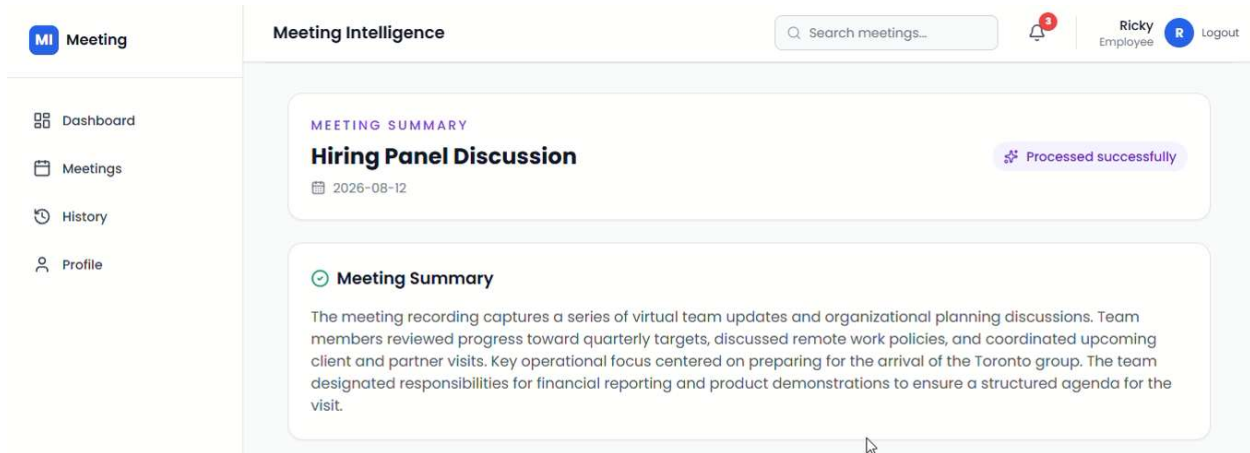


Figure 5.3. Meeting Summary

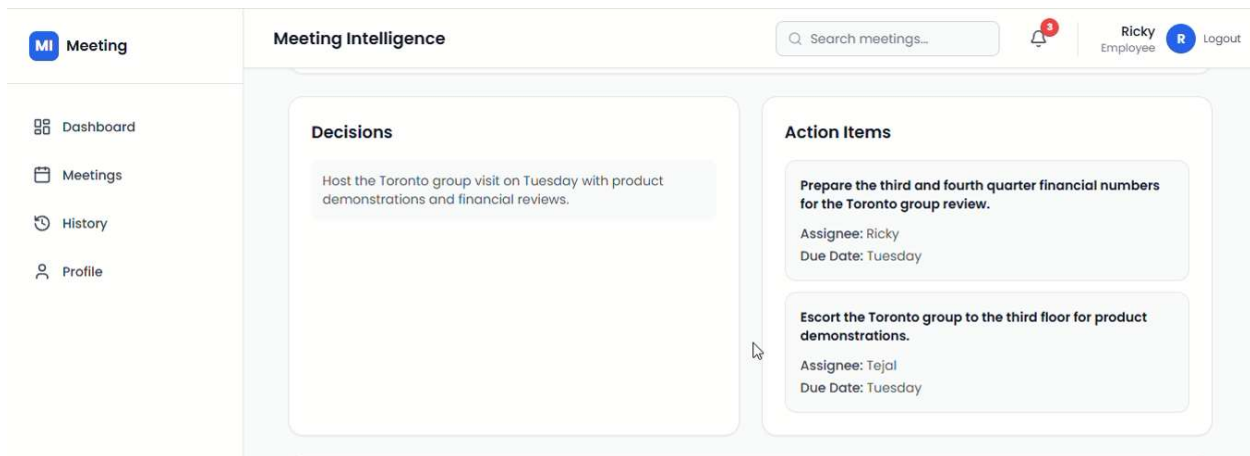


Figure 5.4. Action Items

5.8 FUNCTIONAL VALIDATION

Functional testing was carried out to verify the major operations of the proposed meeting intelligence platform. The testing begins by checking whether the React frontend is accessible through `localhost:5173` and whether the Node.js/Express backend is running correctly through `localhost:5000`. The meeting recording upload functionality is then tested using valid audio and video files within the configured upload limit. After a recording is uploaded, the system is checked to verify that the recording reaches the backend and is successfully

processed by the Gemini AI service. The generated response is examined to confirm the availability of the structured meeting information, including the meeting summary, key discussion points, decisions, and action items.

The action-item functionality is further tested using meeting recordings containing task-related discussions. The generated output is checked to determine whether the system identifies the task, assignee, and due date when such information is available in the meeting content. The result-storage functionality is also verified by checking whether the generated meeting result is written to the configured `data/results.json` file. Finally, the result-retrieval and display functionality is tested to ensure that the stored meeting information can be retrieved and presented correctly through the frontend. The functional test cases cover the complete workflow from frontend access and recording upload through AI processing, result storage, retrieval, and structured result display. The final observed status of each test case should be recorded after performing the tests on the implemented system.

5.9 SYSTEM OUTPUTS

The implemented output is a structured meeting result rather than a single unstructured paragraph. The primary output fields are summary, keyPoints, decisions, actionItems, and deadlines. The actionItems structure contains task, assignee, and dueDate. This organization allows the frontend to render the meeting information as separate sections and provides a foundation for later task-oriented functionality.

Output field	Description	Current implementation status
summary	Overall summary of the	Implemented

	meeting	
keyPoints	Important discussion points	Implemented
decisions	Decisions identified from the meeting	Implemented
actionItems	Action-oriented tasks extracted from the meeting	Implemented
actionItems.task	Task description	Implemented as part of actionItems
actionItems.assignee	Responsible employee/person	Implemented as part of actionItems
actionItems.dueDate	Due date associated with a task	Implemented as part of actionItems
deadlines	Deadline information identified from the meeting	Implemented

Table 5.2 Input/output description of implemented AI result

5.10 IMPLEMENTATION DISCUSSION AND OBSERVED ISSUES

The following issues should be treated as implementation considerations rather than measured performance failures. The current project information identifies a 25 MB upload limit and temporary local storage. In addition, the use of an external AI API means that processing depends on network availability and the behavior of the configured model/API. The quality of extracted information may also vary with recording quality, background noise, overlapping speech, ambiguous references, and

whether participants explicitly state names or dates. These factors should be evaluated empirically rather than assigned invented accuracy values.

Issue / consideration	Possible effect	Corrective consideration
25 MB upload limit	Large recordings may not be accepted	Increase or redesign upload handling after testing larger media.
Network dependence	AI processing requires access to Gemini	Add retry handling, clearer failure states, and offline processing in future work.
Recording noise	Speech/content may be harder to interpret	Evaluate preprocessing or dedicated ASR in future work.
Overlapping speech	Multiple simultaneous speakers may reduce clarity	Evaluate speaker-aware processing and diarization in future work.
Ambiguous names or dates	Assignee/due-date extraction may be uncertain	Add validation and user confirmation workflows.
Local JSON storage	Not suitable for large multi-user enterprise persistence	Migrate to a database for production-scale use.
External LLM dependence	Output can vary across model/API versions	Pin and document model configuration and add structured validation.

Table 5.2. Observed issues and corrective considerations

5.11 LIMITATIONS

- The current system is recording-based and does not provide real-time meeting transcription.
- Direct integration with online meeting platforms is not implemented.
- Speaker identification is not implemented.
- Multilingual transcription is not implemented as a dedicated project feature.
- Email, calendar, and document integrations are not implemented.
- Automatic task reminders are not implemented.
- Employee task-progress tracking is not implemented.
- Enterprise authentication and role-based access control are not described as implemented.
- The current result-storage mechanism is local JSON storage rather than a production database.
- The current upload limit is 25 MB.
- No project-specific accuracy percentage, user study, benchmark score, or large-scale evaluation is reported because no such measurements were supplied.
- The current system uses a hosted AI service, so processing depends on network availability and the service configuration.

5.12 SUMMARY OF CONTRIBUTIONS

The completed phase demonstrates a working application architecture for recording-based enterprise meeting intelligence. The frontend provides the meeting interface, the backend manages recording upload and API communication, Multer provides file handling, the Gemini service performs multimodal meeting analysis, and the result is organized into structured JSON and stored locally. The application therefore

establishes an end-to-end path from meeting recording to structured meeting information.

The main contribution is the practical integration of these components into one workflow. The system does not claim a new speech-recognition algorithm, a new summarization model, or a statistically validated action-item extraction model. Its contribution is an application-level platform that demonstrates how a multimodal LLM can be connected to enterprise meeting management requirements and how the resulting structured information can be presented through a web application

CHAPTER 6 CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The AI-Powered Enterprise Meeting Intelligence and Action Tracking Platform was designed to reduce the effort involved in converting recorded meetings into useful, structured information. The system accepts a meeting audio or video recording through a React and Vite frontend, transfers the recording to a Node.js and Express.js backend, processes the media through the Google Gemini API using `@google/genai`, and stores the generated result in a local JSON file. The result is organized into summary, key discussion points, decisions, action items, assignees, due dates, and deadlines.

The architecture demonstrates a clear separation between the presentation layer, backend API, AI-service integration, file-processing layer, and result-storage layer. This structure makes the current implementation understandable and provides a practical foundation for later enterprise capabilities. The project also addresses the real-world problem of employees being unable to maintain complete meeting notes because of travel, noise, network conditions, or the need to concentrate on the discussion itself.

The current phase should nevertheless be interpreted within its defined scope. It is not a real-time meeting assistant, does not directly join online meeting platforms, does not identify speakers, does not send task reminders, and does not provide measured accuracy claims. The value demonstrated by the current implementation is the successful integration of recorded-media ingestion and AI-based structured meeting analysis into a web application. More rigorous evaluation and enterprise-grade deployment remain important areas for future development.

6.2 FUTURE WORK

The platform can be extended in several directions. These extensions should be treated as planned future work and not as features of the current implementation.

- Real-time meeting transcription can be introduced so that meeting content is processed while the meeting is taking place.
- Online meeting integration can connect the platform with meeting systems so that recordings or transcripts are acquired automatically.
- Offline meeting support can reduce dependence on continuous network connectivity by introducing local or edge processing options.
- Multilingual meeting transcription can extend the platform to organizations operating across multiple languages.
- Speaker identification can associate statements and action items with individual participants.
- Email and document integration can allow meeting results to be distributed or combined with related business material.
- Calendar integration can connect due dates and scheduled follow-up activities with organizational calendars.
- Task reminders can notify responsible employees about approaching deadlines after explicit user or administrator configuration.
- Employee task-progress tracking can extend action items into a complete task lifecycle.
- Intelligent meeting search can allow users to search previous meetings for decisions, topics, people, and action items.
- Enterprise authentication can introduce secure user identity, access control, and organization-level management.

- Cloud deployment can provide centralized storage, scalable processing, and access across organizational devices.
- Larger-scale evaluation can measure summary quality, decision extraction, action-item extraction, due-date extraction, latency, reliability, and user satisfaction using a defined benchmark and human-annotated test set.
- Structured validation and confidence handling can be added so that uncertain AI outputs are highlighted for human confirmation instead of being treated as equally reliable.

A particularly important future direction is evaluation. The present report deliberately avoids invented accuracy percentages. A future evaluation should define a representative meeting dataset, establish human-labelled reference summaries and action items, specify evaluation metrics, and report results separately for summary quality, decision extraction, action-item extraction, assignee identification, and due-date extraction. End-to-end latency should also be measured across upload, AI processing, storage, and frontend rendering rather than measuring only one internal stage.

REFERENCES

- [1] J. Carletta, S. Ashby, S. Bourban, M. Flynn, M. Guillemot, T. Hain, J. Kadlec, V. Karaiskos, W. Kraaij, M. Kronenthal, G. Lathoud, M. Lincoln, A. Lisowska, I. McCowan, W. Post, D. Reidsma, and P. Wellner, “The AMI Meeting Corpus: A Pre-announcement,” in *Machine Learning for Multimodal Interaction, Second International Workshop, MLMI 2005, Revised Selected Papers, Lecture Notes in Computer Science*, vol. 3869, Springer, 2006, pp. 28–39, doi: 10.1007/11677482_3.
- [2] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” arXiv:2212.04356, 2022.
- [3] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” in *Advances in Neural Information Processing Systems* 33, 2020.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems* 30, 2017.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, vol. 1, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.
- [6] M. Zhong, D. Yin, T. Yu, A. Zaidi, M. Mutuma, R. Jha, A. H. Awadallah, A. Celikyilmaz, Y. Liu, X. Qiu, and D. Radev, “QMSum: A New Benchmark for Query-based Multi-domain Meeting Summarization,” in *Proceedings of the 2021*

Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2021.

[7] Y.-N. Chen and D. Hakkani-Tür, “AIMU: Actionable Items for Meeting Understanding,” in Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16), 2016, pp. 739–743.

[8] M. T. R. Laskar, X.-Y. Fu, C. Chen, and Shashi Bhushan TN, “Building Real-World Meeting Summarization Systems using Large Language Models: A Practical Perspective,” in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track, 2023, pp. 343–352, doi: 10.18653/v1/2023.emnlp-industry.33.

[9] F. Kirstein, T. Ruas, R. Kratel, and B. Gipp, “Tell me what I need to know: Exploring LLM-based (Personalized) Abstractive Multi-Source Meeting Summarization,” in Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, 2024, pp. 920–939, doi: 10.18653/v1/2024.emnlp-industry.69.

[10] Gemini Team, “Gemini: A Family of Highly Capable Multimodal Models,” arXiv:2312.11805, 2023.

[11] Google, “Gemini API: Audio Understanding,” Google AI for Developers, technical documentation, accessed September 2026.

[12] Google, “Gemini API: Video Understanding,” Google AI for Developers, technical documentation, accessed September 2026.